Geavanceerde computerarchitectuur
Lab sessions

(JLI1QS)

Gheysens Jonathan & Mats Mortier

March 28, 2024



Class: M1EIes

Contents

1 Session 1 - Getting Acquainted with GPU Programming

2 Session 2 - Race conditions

3 Session 3 - Warps, thread divergence, and striding
- 3.1 Warps . . . .
- 3.2 Thread divergence . . . .
- 3.3 Some pointers - nonobligatory . . . .
  - 3.3.1 Implement the inverse kernel and use the provided image load/save function to invert an image. You can use as many threads as there are input elements. . . . .
  - 3.3.2 Implementation of striding in the kernel . . . .
  - 3.3.3 Second kernel where the R component is processed according to $x = (x\%25) \cdot 10, x \in (100, 200)$ . . . .

4 Session 4 - Memory management
- 4.1 Part 1 - Coalesced memory access . . . .
- 4.2 Part 2 – Global, shared, and constant memory . . . .

5 Session 5 - Dynamic parallelism and asynchronous operation
- 5.1 Dynamic parallelism . . . .
- 5.2 Asynchronous operation . . . .

1 Session 1 - Getting Acquainted with GPU Programming

In this lab session, our primary objective was to familiarize ourselves with GPU programming. Unlike CPU programming, which prioritizes sequential thread execution, GPU programming excels in executing multiple threads in parallel. Our focus was on understanding the structure of a program and how it can be optimized for GPU execution.
We began by examining how a program can be broken down into smaller sequential tasks, or threads. To expedite execution, these threads can be run in parallel on the GPU. Threads are organized into blocks, with each block being processed by a distinct GPU core. In this session, our task was to reverse the order of elements in an array.
Initially, we attempted this using only one block containing nine threads. With this setup, all threads had to execute sequentially, leading to nine execution cycles. Next, we divided the threads into three blocks, each containing three threads. This allowed three threads to execute simultaneously, reducing the total execution cycles to three. This can be seen in figure 1.
Finally, we further divided the threads into nine blocks, with one thread per block. This approach necessitated only a single execution cycle, resulting in significant time savings. Although the impact was minimal in this particular case, the advantage would become more pronounced with a substantial increase in the number of threads to be executed.

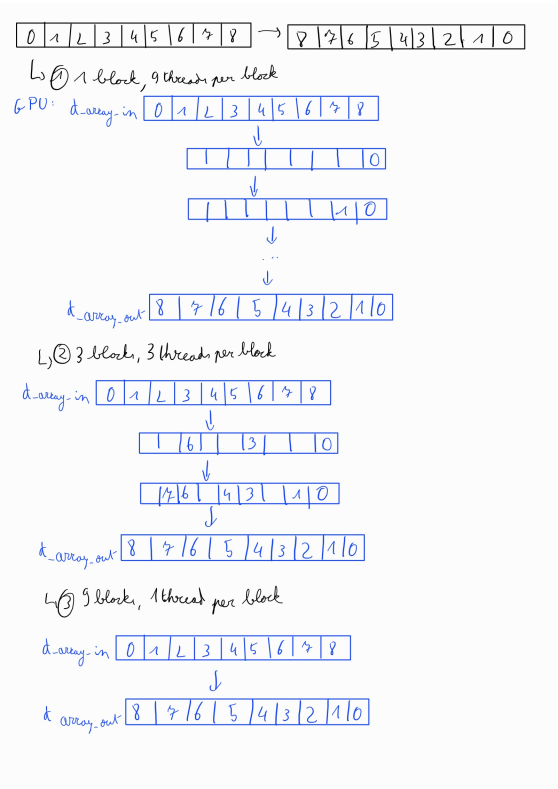


Figure 1: Illustration of blocks and threads

2 Session 2 - Race conditions

During the second lab session we had a look on race conditions and how to avoid them when GPU programming. We did a max detection of the elements of an array using three different methods. The race conditions in this assignment can occur when multiple threads attempt to access the array elements, but the threads of the previous step aren't finished yet. This can be illustrated by figure 3. When one of the threads in the first row hasn't finished yet and the threads of the second step would already start executing, this could give incorrect results. So after each row in figure 3 synchronization must be done to avoid this.
Fist we used a nested for loop on the CPU. For this method we used an extra variable where we stored the maximum value. We initialised this by setting this to the value of the first element of this array. Then using the for loop, we checked if each element of the array greater is then the maximum value variable. If so, we updated the maximum value to the value of the array element value. With this case no race conditions occur.
Secondly, we used a GPU kernel using the atomic max operation. An atomic operation gets executed without interruption.Threads get sequential access to a common resource. Because they have sequential access, there can be no interference and thus, as a result, no race conditions can occur. With the atomic max operation we had a time of 439 μs on an array of size 30000. just for comparison, the nested loop had and time of 2051 μs to find the maximum. So the atomic operations on a large array on the GPU are faster than a nested loop on a large array on the CPU.
Finally, we also used a GPU kernel using reduction. How we will exactly do this can be seen in figure 3, but instead of taking the sum of the elements, we take the maximum value of the two elements. More precisely, we loop over the array with a step size of 1 and search for the maximum value of two elements. The maximum value is placed in source 1 (see figure 2). Before we can continue and loop over the array with a greater step size, we have to synchronize threads first. Otherwise, some threads of the steps could not be finished yet and so we wouldn't get the correct the result. This is what we call race conditions and should be avoided. After the synchronisation we continue looping over the array, but each time with a greater step size as seen in figures 3 and 2. We stop looping when the step size is smaller then log_2(array_size).

Q: Is the result correct?
To fill the array, a loop going from 0 to aray size minus 1 was used each time. With nested loop and the atomic max, array of size 30 000 was used and a maximum was obtained of 29999. This is correct because the largest value contained in the array was 300000 - 1 = 29999.
With the GPU kernel, we had an array of size 2048. From this we got a maximum of 2047, this is correct as well!
Q: How does time increase as a function of input size?
With a very small array we saw that the CPU was always faster than the GPU, but with a huge array the GPU was much faster than the CPU. We can conclude from this that for small array operations the CPU is better but from that working with large arrays we better switch to the GPU. This can be declared by having a look at how CPU and GPU programming works, but this is already explained in section 1 (report of lab session 1).
Q: What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?
The first option is to use an atomic max operation to get the maximum of an array larger than 2048 elements. The atomic max operation is executed on the two maximum elements of each block (original array is splitted in two or more blocks). Secondly, we can also take the maximum of more then two elements within our kernel reduction method. This allows us to use larger

input arrays.

Q: Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

As mentioned in the answer above , we can split up the original array, so we can process a maximum amount of data elements with the given number of threads. After this first step we can reuse the kernel function to get the maximum element of all the maximum values which were stored in a new array.

Q: How would you tackle a 2D array?

For a 2D array we would take the maximum value of each column and then place them in a new 1D array. Tthis new array can then be used to find the max as in the previous examples for 1D-arrays. +

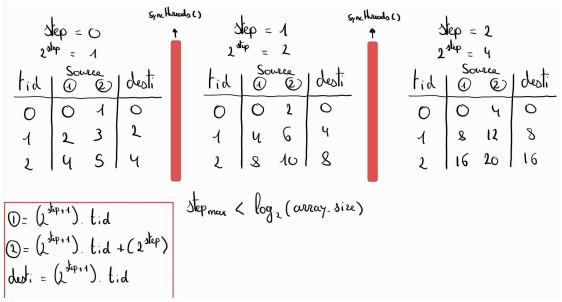


Figure 2: Illustration of reduction-method

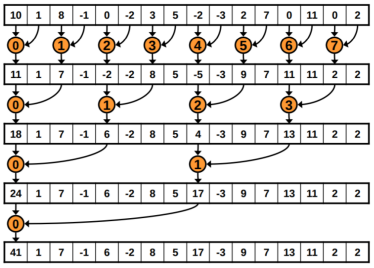


Figure 3: Illustration of array reduction

3 Session 3 - Warps, thread divergence, and striding

3.1 Warps

A warp is a group of 32 threads. These threads are grouped together for efficient execution of the same instruction simultaneously but operate on different data. So if we want to calculate the number of warps, we can do this with the following function: Number of warps = $(\frac{Threadsperblock}{32})\cdot$ number of blocks

In this first section, we are going to implement a kernel that inverts a image. To invert an image, we need to calculate the x = 255 - x for each color component.
When we use 1 thread, we become a execution time of 111854 μs (see figure 4). In this case, 111854 warps are created with a lot of GPU cores running idle. Each block gets a full warp assigned, but with only 1 thread per block the other 31 threads aren't used but still assigned for this block. When we are incrementing the amount of threads, then the execution time drops, because less warps are needed. When we are using 192 threads, we have a minimal excecution time of 71850 μseconds. This is the limit, because the GPU we used during this lab has only 192 cores and this is also the maximum of threads we can run in parallel.

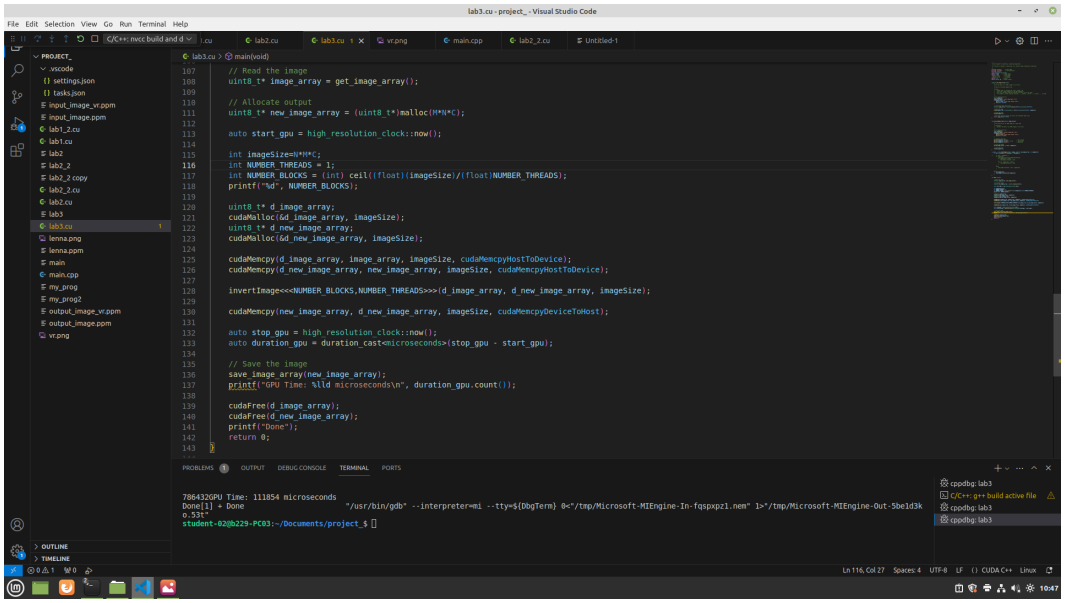


Figure 4: Excecution time 1 thread

3.2 Thread divergence

One part of the warp may have higher complexity and thus longer computation time. Because the other parts of the warp become free, they temporarily no longer contribute to the processing of the data but still possess a core.... We can solve this by first starting to split the incoming array in the code by R, G and B values. As seen in the task, the computational cost is higher for the R values than for the G and B values. Therefore, we are actually going to create a new

second kernel in order to compute the values separately so that no thread divergence occurs and the GPU cores are used optimally.

3.3 Some pointers - nonobligatory

3.3.1 Implement the inverse kernel and use the provided image load/save function to invert an image. You can use as many threads as there are input elements.



Figure 5: Image lenna



Figure 6: Inversed image lenna

3.3.2 Implementation of striding in the kernel

Striding occurs when the matching between threads and elements is not 1 to 1. This can cause the need to compute multiple elements in one thread. We can do this by checking which iteration we are in (starting from 0), multiplying this by the size of the grid of threads and then adding this to the initial index of the element. We do this until the new index is 1 less than the length of the incoming array. This was then implemented on an array with 16 elements and 12 threads. (See figure 7.) This involved adding 1 to each element using only 12 threads for all the 16 elements and obtaining the correct result.

In figure 8, we can see that the execution time is decreasing with the amount of threads. So we can conclude that if we use more threads, the execution will happen faster then if we use just a few threads.



Figure 7: Example striding

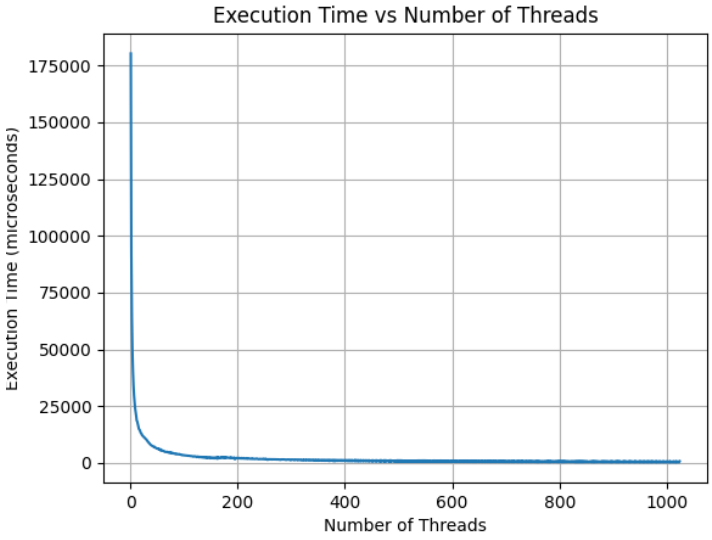


Figure 8: Execution time vs number of threads

3.3.3 Second kernel where the R component is processed according to $x = (x\%25) \cdot 10, x \in (100, 200)$



Figure 9: image lenna



Figure 10: second inversed image lenna

After this we made a third kernel, which instead of processing RGBRGB...RGB operates on an input format of RR...RGGG...GBB...B.
If we inverse the image with the additional conditions, we had a time of 76435μs. Now with the reformation of the input array we compute the R seperatly of the G and B (who need less computation). Each warp consist now of threads with equal computation costs, which reduces the GPU cores running idle. In total, this can give us a significant time gain.

4 Session 4 - Memory management

4.1 Part 1 - Coalesced memory access

As mentioned in earlier parts, threads are grouped in warps. Threads in the same warp mostly access adjacent memory addresses. Coalesced memory access allows us to combine multiple memory accesses into a single memory transaction. This reducess the number of required transactions and improves the execution time of a GPU program.



Figure 11: image lenna

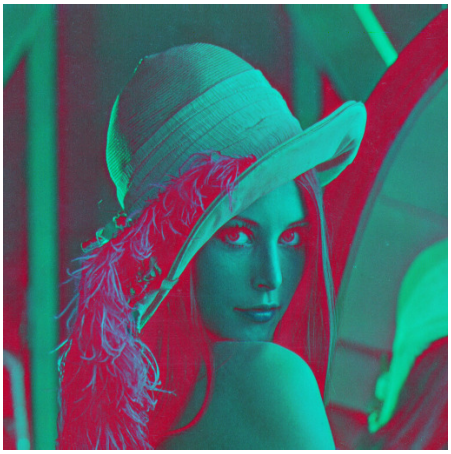


Figure 12: R-value inversed on lenna

As seen in figure 13, coalesced memory acces speeds up the execution in comparison with un-coalesced memory access. Even for smaller grid-sizes we can see an advantage by using coalesced memory access.

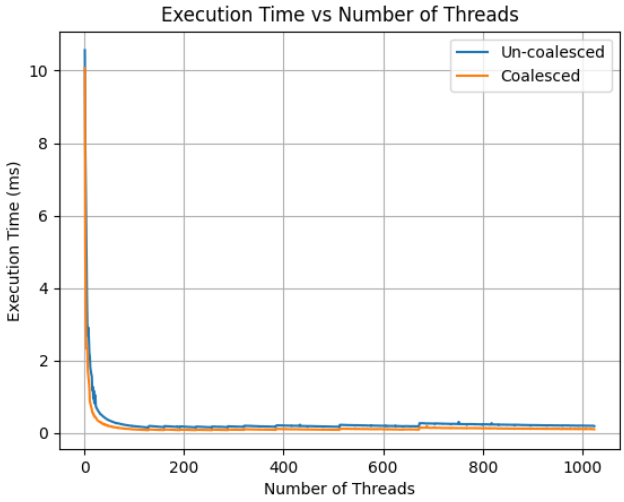


Figure 13: execution time un-coalesced vs coalesced

4.2 Part 2 – Global, shared, and constant memory

As second part of this lab session, we did matrix multiplications using different types of memory. Firstly, we used global memory to store our matrices. This memory is large and can be accessed by all functions, but is slow in comparison with other types. Secondly, we stored our matrices in shared memory. Shared memory is a small but very fast type of memory that is shared among threads within the same block. As third part, we also tried storin a matrix in constant memory. This type of memory is located off-device and optimized for low-latency read operations. Constant memory is typically used to store data that is constant throughout the execution of a kernel. More concrete, we started by using 2x2 matrices and in figure 14 the input matrices A and B are visible. Matrix multiplication is a mathematical operation where the elements of rows in one matrix are multiplied with corresponding elements of columns in another matrix, and the results are summed to generate a new matrix. So in our case, the matrix multiplication can be done as following: 3*3 + 6*6 = 45, 3*5 + 6*2 = 27, 7*3 + 5*6 = 51, 7*5 + 5*2 = 45. These values are equal to those we get in our program, so we can conclude our matrix multiplication is done correctly.

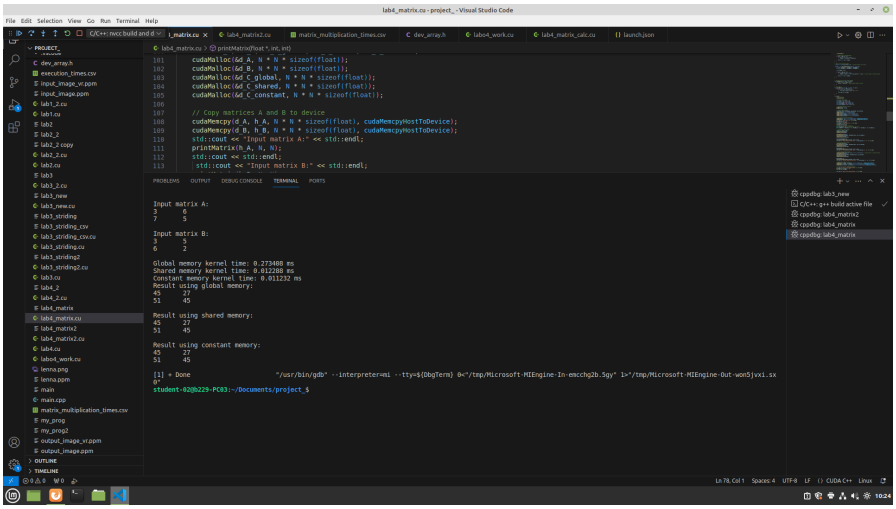


Figure 14: Verification matrix multiplication

By increasing the size of the matrices, we observe constant memory is significantly slower then shared and global memory. The results can be found in figure 15. When we want to continue using larger matrices, we need to split up the original matrix in smaller matrices of 32x32 elements, because of the limited size of the shared memory (mostly 16-48kbytes). As visible in figure 16, shared memory becomes faster and faster when going to a matrix size of 256x256 elements.

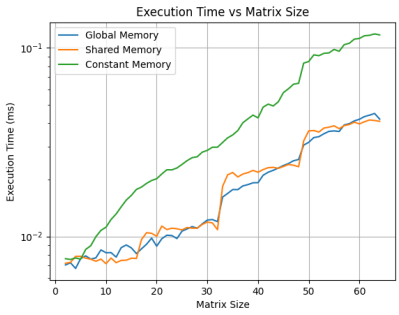


Figure 15: Execution time matrix size until 64x64

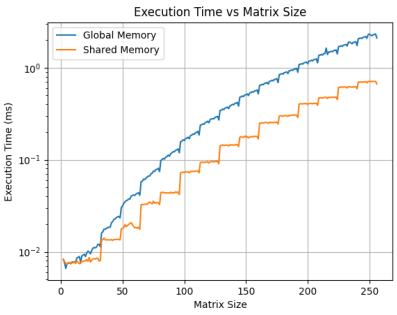


Figure 16: Execution time shared vs. global

5 Session 5 - Dynamic parallelism and asynchronous operation

5.1 Dynamic parallelism

Dynamic parallelism refers to the capability of GPUs to create and manage parallel tasks or threads on-the-fly within a kernel. For example, the GPU can spawn a kernel from within a kernel, enabling more flexible and complex program structures. This isn't executed during the lab sessions, because this requires at least compute capability 3.5, but the PCs in the lab only have 3.0.

5.2 Asynchronous operation

In GPU programming, asynchronous operation means that the GPU can execute independently at the same time of the CPU. Using this principle, we can maximize the CPU & GPU utilization without waiting for each other. This can reduce the execution time of a program. In the lab, we used 4 input arrays and 4 kernel functions to find the sum, product, minimum and maximum of these arrays. Firstly, we executed the full program synchronously: create an array (CPU), execute kernel function (GPU), create an array (CPU) ... Before executing the next step, we wait for the current step to finish. Secondly, we also tried doing this asynchronously. In stead of waiting for the GPU to finish, we already start creating the next array on the CPU. The total execution time for different array sizes are visible in figure 17.

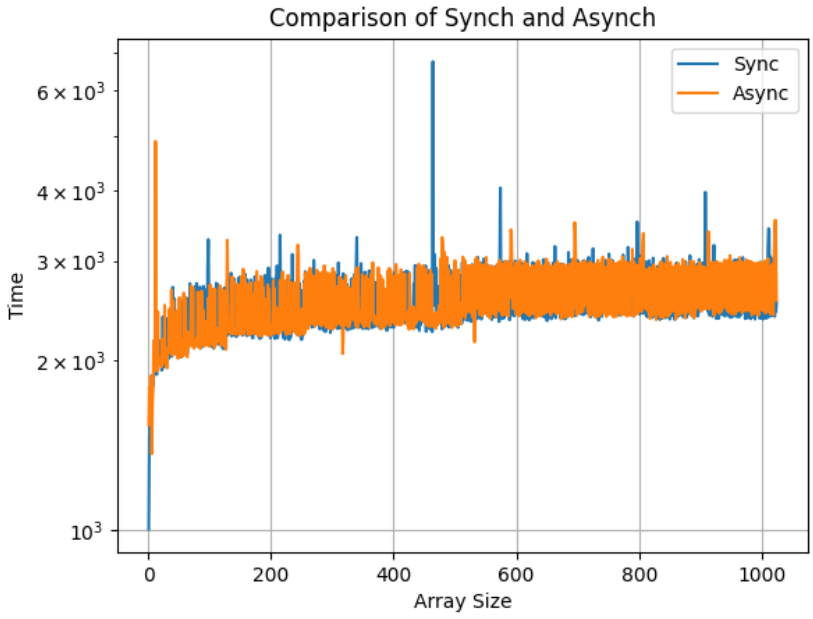


Figure 17: Synchronous vs Asynchronous Operation

Figure 17 doesn't give us a proper conclusion as both graphs are almost equivalent. To understand why the graphs are similar, we measured the execution time of the CPU and GPU

separately. In figure 18 we observe that the GPU execution time is much larger than the CPU time. Using asynchronous operation will not give us a big advantage, because the CPU execution time can almost be neglected. The CPU times are so small, that we could say that the CPU time can be equal to a measurement error on the total execution time.

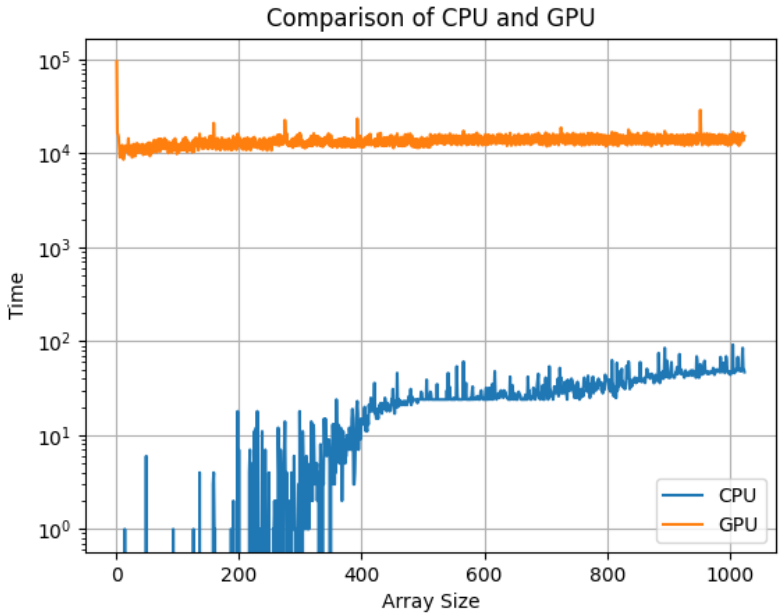


Figure 18: CPU time vs GPU time

Based on figure 18, it's not immediately evident that asynchronous operations provide a significant time advantage. This is due to our system operating very briefly on the CPU, resulting in only a marginal time benefit. However, it becomes apparent that if we were to execute a longer operation on the CPU and then proceed asynchronously, we could indeed gain a noticeable time advantage.